

onet-v2 Technical Reference

St. Nicholas Orthodox Church Website

orthodox.net

March 2026

Abstract

This document is the technical reference for **onet-v2**, the modernization of orthodox.net, the website of St. Nicholas Orthodox Church in McKinney, Texas. It covers the production server architecture on Inoa/Orthodox Internet Services, Apache URL routing and **.htaccess** configuration, the legacy content migration system (PHP fetchers, JavaScript shell scripts, and rewrite rules), frontend and JavaScript architecture, external API integrations (Ghost CMS, YouTube, Google Calendar), Python utility scripts, local development setup, deployment procedures, and operational maintenance. The document is intended for senior developers, DevOps engineers, or new maintainers taking over the project.

Contents

Executive Summary	4
1 System Overview	5
1.1 The Legacy Site and Origins	5
1.2 Goals of onet-v2	5
1.3 Repository	6
1.4 Technology Stack	6
1.5 External Service Dependencies	7
2 Production Server Architecture (Inoa)	8
2.1 Server Directory Layout	8
2.2 The Production .htaccess	8
2.2.1 Legacy Folder Shell Routing (lines 7–84)	8
2.2.2 New Content Directory Routing (lines 87–90)	10
2.2.3 Media CDN Redirect (lines 93–98)	10
2.2.4 URL Normalization (lines 103–140)	10
2.2.5 Server Configuration (lines 142–167)	10
2.3 Adding a New Legacy Folder Conversion	11

3	.htaccess Management: Local vs. Production	11
3.1	Why They Differ	11
3.2	File Inventory	12
3.3	Critical Operational Rule	12
3.4	Verifying the Production <code>.htaccess</code>	12
4	Legacy Content Migration System	13
4.1	Request Flow (Production)	13
4.2	PHP Content Fetchers	14
4.3	JavaScript Shell Scripts	15
5	Frontend Architecture	16
5.1	HTML Template Structure	16
5.2	CSS Architecture	16
5.2.1	Skeleton Framework	16
5.2.2	Custom Stylesheets	16
5.2.3	Design Tokens	17
5.2.4	Responsive Breakpoints	17
5.3	JavaScript Architecture	17
5.3.1	Configuration	18
5.3.2	Page-Level Cache Busting	18
5.3.3	JavaScript File Reference	18
6	Synaxarion (Saints & Readings)	19
6.1	Data Source	19
6.2	Data Files	20
6.3	Client-Side Loading (<code>synaxarion.js</code>)	20
6.4	Maintenance	20
7	Backend API Reference	21
7.1	Base URL	21
7.2	Endpoints	21
8	Python Utilities	21
8.1	Content Generation Scripts	21
8.2	Template and Styling Scripts	22
8.3	Legacy HTML Processing	22
8.4	File Management and Conversion	23
8.5	Gleanings Utilities	23
8.6	NLP Utilities	24
8.7	Dependencies (<code>requirements.txt</code>)	24
9	Local Development	24
9.1	Setup	24
9.2	Running Locally	25
9.2.1	Option A: PHP Built-in Server (Recommended)	25
9.2.2	Option B: VSCode LiveServer	25
9.2.3	Option C: Local Apache (Full Simulation)	25
9.3	Local vs. Production Differences Summary	25

10 Production Deployment	26
10.1 Standard Deployment	26
10.2 Template Updates	26
10.3 Calendar Data Refresh	26
11 Security	26
11.1 Content Security Policy	26
11.2 Directory Traversal Prevention	27
11.3 HTTPS Enforcement	27
11.4 Credentials and Secrets	27
12 Character Encoding	27
13 Cache Busting Strategy	28
14 Key DOM Element IDs	28
15 Troubleshooting	29
15.1 “Not Found” in Saints & Readings Section	29
15.2 Legacy Content Not Rendering	29
15.3 Backend API Errors	29
15.4 Changes Not Appearing After Deploy	29
16 Repository Files Reference	30
16.1 Git-Ignored Files	30
16.2 Documentation	30
17 Handoff Checklist	30
A Project History by Milestone	32
A.1 Phase 1: Initial Setup and Infrastructure (Jul–Aug 2023)	32
A.2 Phase 2: Dynamic Content Integration (Aug–Oct 2023)	32
A.3 Phase 3: CMS and Blog System (Dec 2024–Feb 2025)	33
A.4 Phase 4: Legacy Folder Migration System (Jun–Aug 2025)	33
A.5 Phase 5: API Migration and Stabilization (Nov–Dec 2025)	34
A.6 Phase 6: Documentation and Maintenance (2026–Present)	34
B Document Revision History	35

Executive Summary

orthodox.net is the website of St. Nicholas Orthodox Church in McKinney, Texas. The original site was built in the early 2000s as a traditional LAMP stack (Linux, Apache, MySQL, PHP) and accumulated thousands of static HTML pages over two decades. The `onet-v2` project, hosted at <https://github.com/GZancewicz/onet-v2>, is a progressive modernization of this site that replaces legacy pages one at a time without modifying the originals.

Architecture. The site runs on the same Inoa/Orthodox Internet Services shared hosting (cPanel). A single `.htaccess` file at `/public_html/` routes requests: shell routing rules intercept legacy URLs and serve them through modern templates; a fallback rule checks for new content in `/new/onet-v2/content/`; anything else is served from legacy `/public_html/`. The frontend is vanilla HTML5, CSS3 (Skeleton framework), and JavaScript — no build system, no frameworks. A Python Flask API on Render.com integrates external content from Ghost CMS, YouTube, and Google Calendar.

Legacy Migration. The most architecturally significant subsystem. For each of 24 legacy content categories (sermons, articles, scripture, etc.), an Apache rewrite rule redirects to a shell HTML page. JavaScript in the shell page calls a PHP fetcher script, which reads the legacy HTML file from disk, extracts the content between marker comments, converts from Windows-1252 to UTF-8, and returns it. The JavaScript then cleans Microsoft Word artifacts and injects the result into the modern page template.

Key Operational Details. The production `.htaccess` is *not* deployed via `git pull` — it must be manually maintained on the server at `~/public_html/.htaccess`. The local version uses relative paths; production uses absolute paths. The “Today’s Saints & Readings” section depends on pre-generated static files from `fetch_calendar.py`, which must be re-run every 90 days.

Technology Stack. HTML5, CSS3, vanilla JavaScript (frontend); Apache with `mod_rewrite` and minimal PHP (server); Python Flask on Render.com (backend API); Ghost.io (CMS); Python 3 utility scripts (maintenance). Version control via GitHub with deployment by `git pull` on the production server.

This Document. This technical reference serves as both an operational guide and a handoff document. It covers the production server architecture, URL routing, legacy migration system, frontend and JavaScript architecture, backend API endpoints, Python utilities, local development setup, deployment procedures, security considerations, and troubleshooting. Appendix A provides a project history by milestone; Appendix B records this document’s revision history.

Repository: <https://github.com/GZancewicz/onet-v2>

Production: <https://www.orthodox.net>

Backend API: <https://github.com/GZancewicz/onet-flask-backend>

1 System Overview

onet-v2 is the frontend repository for <https://www.orthodox.net>, the website of St. Nicholas Orthodox Church in McKinney, Texas.

1.1 The Legacy Site and Origins

The original orthodox.net website was built as a traditional LAMP stack (Linux, Apache, MySQL, PHP) site in the early 2000s. Over two decades, it accumulated thousands of static HTML pages across dozens of content categories — sermons, articles, scripture readings, catechism materials, saints’ lives, and more. Many of these pages were authored in Microsoft Word and exported to HTML, resulting in inconsistent encoding (Windows-1252), heavy inline styling, and non-semantic markup.

The site was maintained by clergy using a combination of Perl scripts for batch updates and FTP for file uploads, hosted by Orthodox Internet Services (Inoa) on shared cPanel hosting. There was no version control, no templating system, and no separation of content from presentation.

1.2 Goals of onet-v2

The onet-v2 project was initiated to modernize the site while respecting several constraints:

1. **Progressive migration:** New pages replace legacy pages one at a time. The legacy site must remain fully functional during the transition, with no modifications to existing legacy pages.
2. **Minimal technical requirements:** The site should be maintainable by volunteers with basic web development skills. No complex build systems, no server-side frameworks, no databases.
3. **Same hosting:** Continue using the existing Inoa/OIS LAMP hosting. No infrastructure migration required.
4. **Modern presentation:** Responsive design, mobile-friendly, accessible, with a consistent visual identity across all pages (new and legacy).
5. **Content management:** Integrate with a headless CMS (Ghost.io) for blog posts and articles, while keeping legacy content accessible through the same URL structure.
6. **Version control:** All changes managed through GitHub with pull request review, replacing the previous FTP-based workflow.
7. **No disruption to clergy workflow:** The upgrade process and ongoing maintenance must not interfere with the church’s day-to-day operations.

A chronological summary of how the project evolved through six development phases is provided in Appendix [A](#).

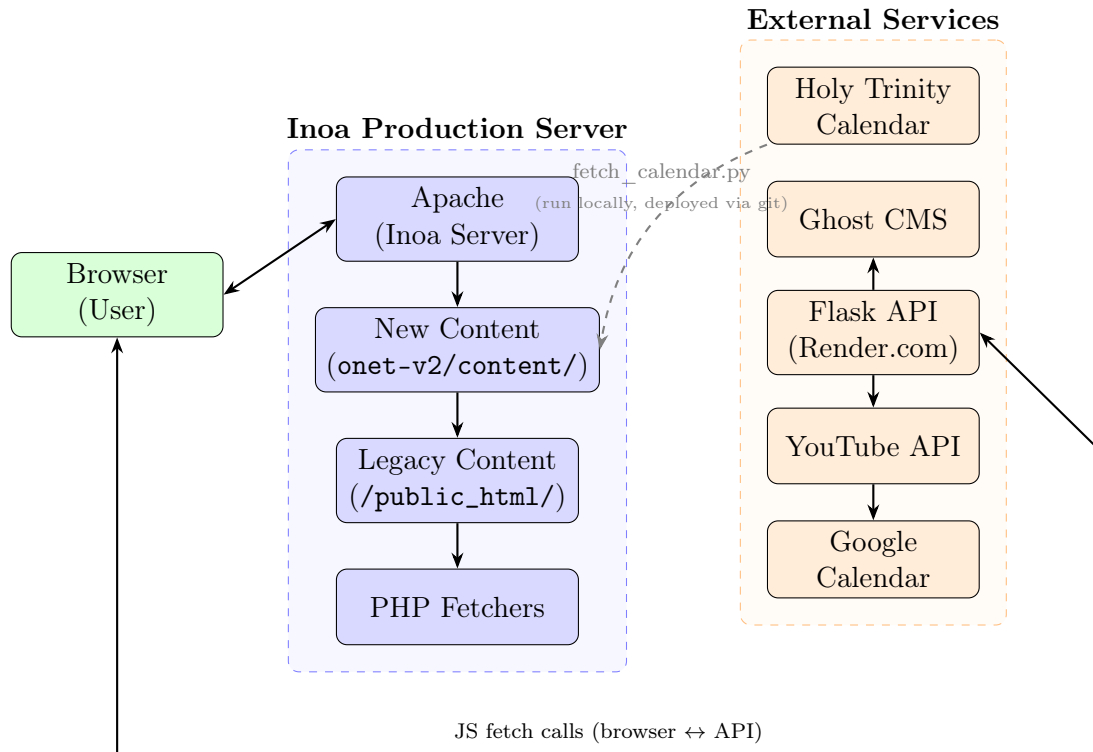


Figure 1: System architecture: how the browser, server, and external services interact.

1.3 Repository

The source code is hosted on GitHub at:

<https://github.com/GZancewicz/onet-v2>

The Flask backend API is a separate repository at:

<https://github.com/GZancewicz/onet-flask-backend>

1.4 Technology Stack

- **Markup/Styling:** HTML5, CSS3 (Skeleton grid framework), Google Fonts (Alegreya), Line Awesome icons
- **Client-Side Logic:** Vanilla JavaScript (no frameworks, no build process)
- **Server:** Apache 2.x with `mod_rewrite`, minimal PHP for content fetcher scripts
- **Backend API:** Python Flask on Render.com (separate repo)
- **CMS:** Ghost.io (headless, API-only)
- **Utilities:** Python 3 scripts for content processing and maintenance
- **Version Control:** Git, deployed via `git pull` on production

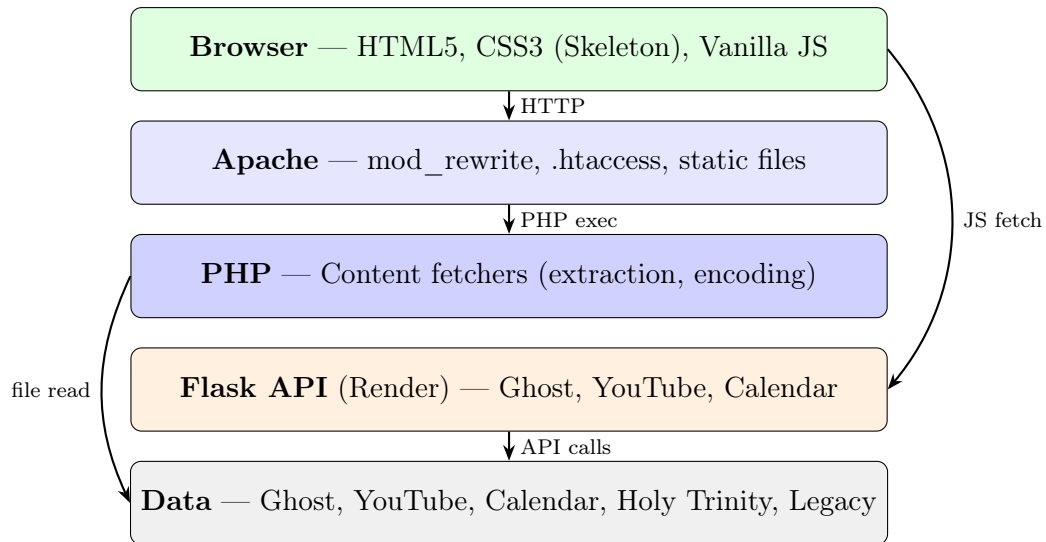


Figure 2: Technology stack layers and data flow between them.

1.5 External Service Dependencies

Service	Purpose
Inoa / Orthodox Internet Services	Production LAMP hosting (cPanel access required)
Render.com	Hosts Flask backend API
Ghost.io	Headless CMS for articles and blog posts
Google Calendar API	Church services and events schedule
YouTube Data API v3	Video content from @orthodoxnet channel
Holy Trinity Orthodox	Daily saints and readings calendar data
Google Analytics	Site analytics (ID: G-D9YEGT19W7)
PayPal SDK	Donation processing

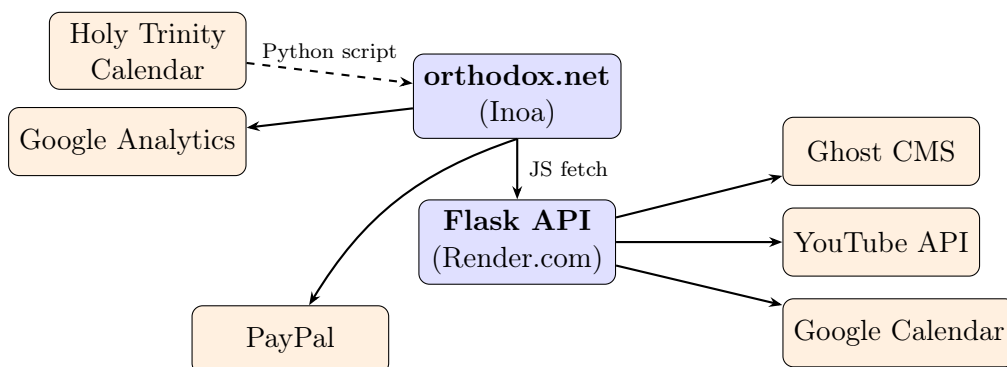


Figure 3: External service dependencies and how they connect to the site.

2 Production Server Architecture (Inoa)

2.1 Server Directory Layout

The production server is hosted by Inoa/Orthodox Internet Services, accessed via cPanel. The directory structure:

```
~/public_html/           # Apache DocumentRoot
.htaccess                # THE master .htaccess (Section 2.2)
articles/, sermons/, ... # ~25 legacy content directories
new/                    # (no .htaccess here)
  onet-v2/              # This repository (git clone)
    content/            # Served content
      index.html        # Homepage
      js/, css/, images/ # Static assets
      calendar/data/    # Saints/readings HTML files
      articles/new/     # Shell pages for legacy content
      *.php             # Content fetcher scripts
    util/               # Python maintenance scripts
    doc/                # Documentation
    apache/             # Reference Apache config files
    legacy/             # Local testing placeholders
```

Key observation: There is **no** `.htaccess` in `/public_html/new/`. All routing is handled by the single `.htaccess` in `/public_html/`.

Figure 4 shows how the `onet-v2` repository fits into the existing LAMP filesystem on both the production server and the local development environment.

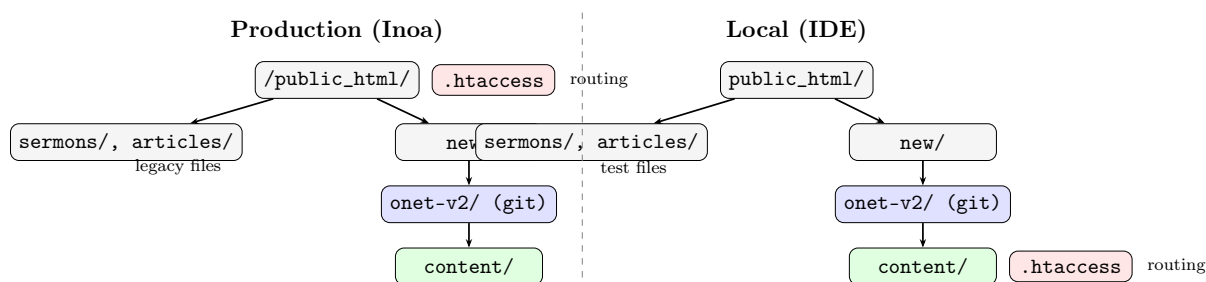


Figure 4: The `onet-v2` repo within the LAMP filesystem.

2.2 The Production `.htaccess`

Location on server: `~/public_html/.htaccess`

Figure 5 illustrates the routing decision process.

This is the single most critical configuration file for the entire site. It is **not deployed** via `git pull` and must be **manually maintained** on the production server. A reference copy is stored in the repository at `.htaccess.production`, but edits to the production file must be made directly on the server via cPanel File Manager or terminal.

The file combines all routing logic into one place. In order of processing:

2.2.1 Legacy Folder Shell Routing (lines 7–84)

These rules intercept requests to legacy content URLs and rewrite them to shell pages that dynamically render the content in the new template. They fire **before** the content directory check.

Pattern: For each legacy content category, requests to `/category/filename.html` (excluding `index.html`) are rewritten to a shell page with an absolute path:

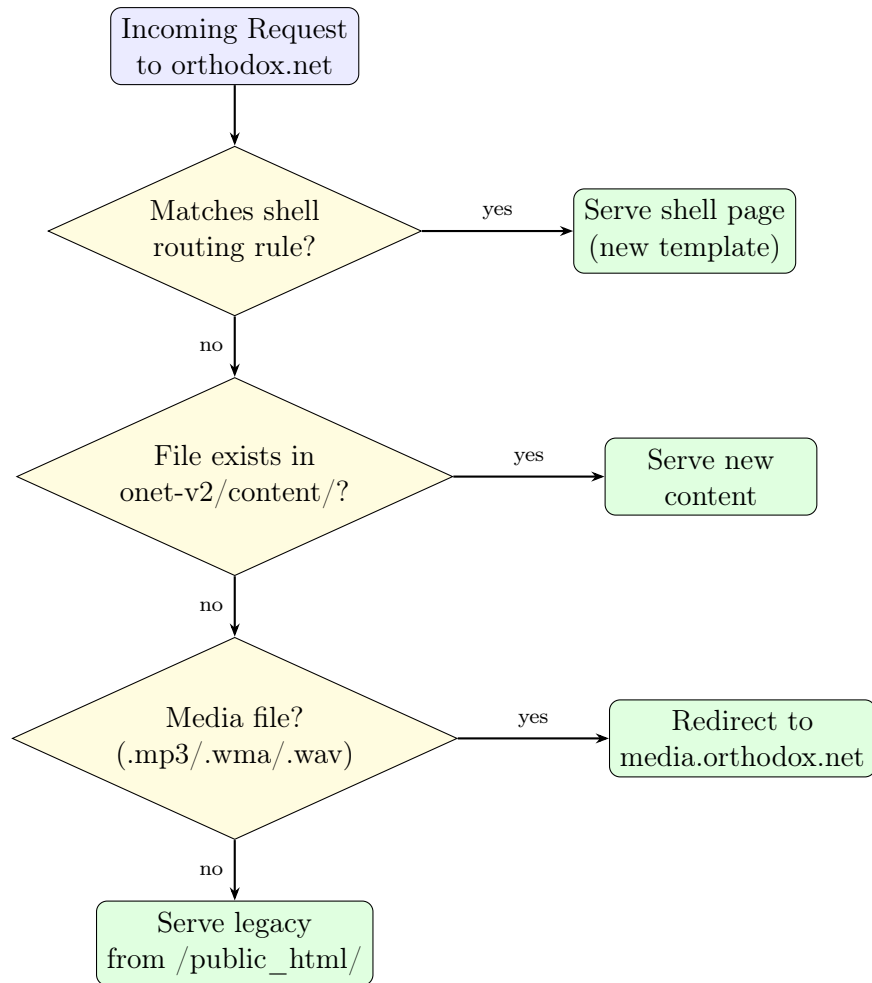


Figure 5: Production `.htaccess` routing decision flow.

```
RewriteRule ^sermons/(?!(index\.html$))([~/]+\\.html)$
/new/onet-v2/content/sermons/new/sermon_shell.html?file=$1 [L,QSA]
```

Note the absolute path prefix `/new/onet-v2/content/` — this is a critical difference from the local development `.htaccess` which uses relative paths (see Section 3).

Blog posts have a special rule:

```
RewriteRule ^posts/([~/]+)/?$ new_post.html [L]
```

Mapped categories (25 total):

URL Path	Shell Page	PHP Fetcher
/articles/	article_shell.html	article_fetcher.php
/sermons/	sermon_shell.html	sermon_fetcher.php
/scripture/	scripture_shell.html	scripture_fetcher.php
/full-voice/	fullvoice_shell.html	fullvoice_fetcher.php
/questions/	questions_shell.html	questions_fetcher.php
/greatlent/	greatlent_shell.html	greatlent_fetcher.php
/pascha/	pascha_shell.html	pascha_fetcher.php
/theophany/	theophany_shell.html	theophany_fetcher.php
/confess/	confess_shell.html	confess_fetcher.php
/fathers/	fathers_shell.html	fathers_fetcher.php

/gleanings/	gleanings_shell.html	gleanings_fetcher.php
/ustav/	ustav_shell.html	ustav_fetcher.php
/services/	services_shell.html	services_fetcher.php
/trebnic/	trebnic_shell.html	trebnic_fetcher.php
/music/	music_shell.html	music_fetcher.php
/menaion-thoughts/	menaion-thoughts_shell.html	menaion-thoughts_fetcher.php
/saints/	saints_shell.html	saints_fetcher.php
/russiannm/	russiannm_shell.html	russiannm_fetcher.php
/links/	links_shell.html	links_fetcher.php
/western-saints/	western-saints_shell.html	western-saints_fetcher.php
/monastic-stories/	monastic-stories_shell.html	monastic-stories_fetcher.php
/poetry/	poetry_shell.html	poetry_fetcher.php
/nativity/	nativity_shell.html	nativity_fetcher.php
/journal/	journal_shell.html	journal_fetcher.php
/posts/*	new_post.html	(via Flask API)

2.2.2 New Content Directory Routing (lines 87–90)

If no shell routing rule matched, check whether the requested file exists under `/new/onet-v2/content/`. If so, serve it from there:

```
RewriteCond %{DOCUMENT_ROOT}/new/onet-v2/content%{REQUEST_URI} -f [OR]
RewriteCond %{DOCUMENT_ROOT}/new/onet-v2/content%{REQUEST_URI} -d
RewriteRule ^(.*)$ /new/onet-v2/content/$1 [L]
```

If the file does **not** exist in the new content directory, Apache falls through and serves from `/public_html/` (legacy content). This is the core mechanism enabling progressive migration.

2.2.3 Media CDN Redirect (lines 93–98)

MP3, WMA, and WAV files are redirected to `media.orthodox.net`:

```
RewriteCond %{HTTP_HOST} ^(www\.)?orthodox\.net [NC]
RewriteRule (.*)\.mp3 http://media.orthodox.net/$1.mp3 [L]
RewriteRule (.*)\.wma http://media.orthodox.net/$1.wma [L]
RewriteRule (.*)\.wav http://media.orthodox.net/$1.wav [L]
```

2.2.4 URL Normalization (lines 103–140)

Forces all URLs to lowercase via character-by-character replacement with 301 permanent redirects. Excludes cPanel DCV validation and ACME challenge paths.

2.2.5 Server Configuration (lines 142–167)

- `ErrorDocument 404 /cgi-bin/404.pl` — custom Perl 404 handler
- `AddType "text/html; charset=windows-1251"` for `.shtml`, `.html`, `.htm` — legacy charset
- Media MIME types for legacy Windows Media formats (ASF, WMA, WMV, etc.)
- PHP handler: `ea-php82` via cPanel
- `ModPagespeed unplugged` — disables Google PageSpeed module

2.3 Adding a New Legacy Folder Conversion

When migrating a new legacy content category:

1. Create the shell HTML page at `content/CATEGORY/new/CATEGORY_shell.html`
2. Create the PHP fetcher at `content/CATEGORY_fetcher.php`
3. Create the JavaScript shell script at `content/js/CATEGORY-shell.js`
4. Add a RewriteRule to the **production** `.htaccess` at `~/public_html/.htaccess` (manually, via cPanel)
5. Add the same rule (with relative paths) to `content/.htaccess` in the repo for local development
6. Commit and deploy the new files via `git pull`

3 .htaccess Management: Local vs. Production

This is one of the most important operational details for this project. The `.htaccess` files that control URL routing are **different between local development and production**, and the production file is **not managed by git**.

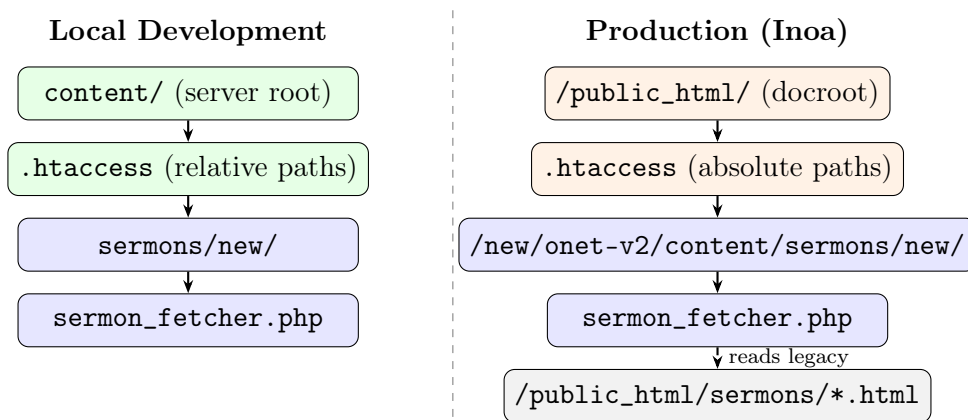


Figure 6: Side-by-side comparison of local vs. production path resolution.

3.1 Why They Differ

On the production server (Inoa), the Apache DocumentRoot is `/public_html/`. The new site content lives several directories deep at `/public_html/new/onet-v2/content/`. Shell routing rules must use absolute paths from the DocumentRoot:

```
# PRODUCTION: absolute path from /public_html/
RewriteRule ^sermons/(?!index\.html$)([/]+\\.html)$
    /new/onet-v2/content/sermons/new/sermon_shell.html?file=$1 [L,QSA]
```

Locally (using `php -S localhost:8000` from the `content/` directory), the server root *is* the content directory, so rules use relative paths:

```
# LOCAL: relative path (content/ is the server root)
RewriteRule ^sermons/(?!index\.html$)([/]+\\.html)$
    sermons/new/sermon_shell.html?file=$1 [L,QSA]
```

3.2 File Inventory

File in Repo	Purpose
content/.htaccess	Local development version. Uses relative paths. Checked into git. Used when running <code>php -S</code> from <code>content/</code> .
.htaccess.production	Reference copy of what should be on the production server at <code>~/public_html/.htaccess</code> . Uses absolute paths. Checked into git but not automatically deployed .
apache/.htaccess	Reference copy of the primary rewrite rules (content routing, CDN, URL normalization). Used as basis for the production file.
apache/.htaccess.inoa	Historical: older Inoa config using symlinks (<code>/onet/</code> , <code>/parent/</code>). Superseded.
apache/.htaccess_v1	Historical: earlier version without content directory routing.
apache/public_html/.htaccess	Reference copy of production <code>.htaccess</code> with both shell routing and content routing combined.
apache/httpd.conf	Local Apache config for macOS (Homebrew). Points <code>DocumentRoot</code> to <code>content/</code> .

3.3 Critical Operational Rule

When you add or modify a shell routing rule, update **both**:

1. `content/.htaccess` in the repo (relative paths, local dev)
2. `~/public_html/.htaccess` on Inoa (absolute paths, via cPanel)

Keep `.htaccess.production` in the repo as a reference copy.
`git pull` does **not** update the production `.htaccess`.

3.4 Verifying the Production .htaccess

The production file includes a test redirect at the top:

```
Redirect 302 /test-htaccess-check /test-htaccess-check-success
```

To verify the `.htaccess` is active, visit <https://orthodox.net/test-htaccess-check> in a browser. If it redirects to `/test-htaccess-check-success`, the file is active.

4 Legacy Content Migration System

The legacy migration system is the most architecturally significant part of the codebase. It allows legacy HTML pages (some dating back 20+ years, many exported from Microsoft Word) to be served through the new site template without modifying the original files.

4.1 Request Flow (Production)

Figure 7 illustrates the complete request lifecycle for legacy content.

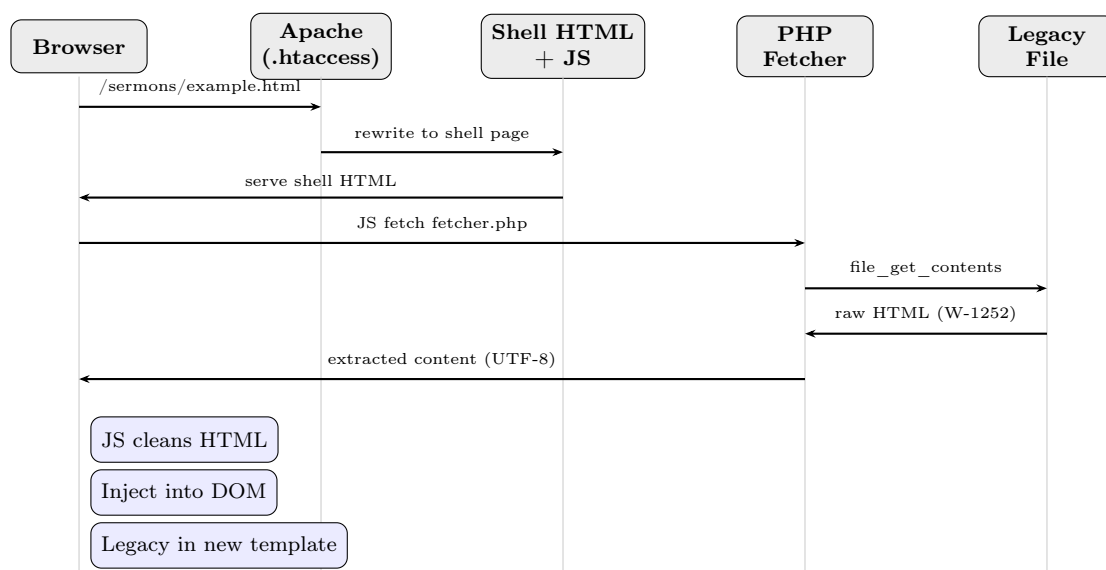


Figure 7: Sequence diagram: legacy content request lifecycle on production.

1. User requests `https://orthodox.net/sermons/example.html`
2. Apache reads `/public_html/.htaccess`
3. Shell routing rule rewrites to `/new/onet-v2/content/sermons/new/sermon_shell.html?file=example.html`
4. `sermon_shell.html` loads `sermon-shell.js`
5. JavaScript extracts `file=example.html` from the query string
6. JavaScript fetches `/sermon_fetcher.php?file=example.html`
7. PHP reads `/public_html/sermons/example.html` from disk
8. PHP extracts content between `<!--BEGIN-CONTENT-->` and `<!--END-CONTENT-->` markers

9. PHP converts encoding from Windows-1252 to UTF-8
10. JavaScript receives HTML, cleans Microsoft Word artifacts, normalizes paths
11. JavaScript injects cleaned HTML into #sermon-placeholder div
12. Result: legacy content displayed within the modern site template

4.2 PHP Content Fetchers

All fetcher scripts follow the same pattern. Using `sermon_fetcher.php` as the canonical example:

Listing 1: `sermon_fetcher.php` (representative pattern)

```

1  <?php
2  header('Content-Type: text/html; charset=utf-8');
3  mb_internal_encoding('UTF-8');
4
5  if (!isset($_GET['file'])) {
6      http_response_code(400);
7      echo "Missing file parameter.";
8      exit;
9  }
10
11 // Security: basename() prevents directory traversal
12 $filename = basename($_GET['file']);
13 $filepath = __DIR__ . '/../public_html/sermons/' . $filename;
14
15 if (!file_exists($filepath)) {
16     http_response_code(404);
17     echo "File not found.";
18     exit;
19 }
20
21 $content = file_get_contents($filepath);
22 // Convert legacy Windows-1252 encoding to UTF-8
23 $content = mb_convert_encoding($content, 'UTF-8', 'Windows-1252');
24
25 // Extract content between markers
26 if (preg_match(
27     '/<!--BEGIN-CONTENT-->[\s\r\n]*(.*?) [\s\r\n]*<!--END-CONTENT-->/is',
28     $content, $matches)) {
29     echo $matches[1];
30 } else {
31     echo $content; // Fallback: serve entire file
32 }
33 ?>

```

Security: All fetchers use `basename()` to strip directory components, preventing path traversal attacks.

Encoding: Legacy files use Windows-1252 encoding. Fetchers convert to UTF-8 before returning to the browser.

Content Markers: Legacy pages use `<!--BEGIN-CONTENT-->` and `<!--END-CONTENT-->` HTML comments to delimit the main content area. If markers are absent, the entire file is returned.

File Path: The relative path `__DIR__ . '/../../../../../public_html/` traverses from `content/` up to the server root. This path is correct on the Inoa production server. Locally, it depends on your directory structure matching the expected layout.

4.3 JavaScript Shell Scripts

Each shell script handles client-side content cleaning. The `article-shell.js` pattern:

Listing 2: Shell script pattern (`article-shell.js`)

```
1 document.addEventListener('DOMContentLoaded', function() {
2   // Extract filename from query string or URL path
3   const urlParams = new URLSearchParams(window.location.search);
4   let file = urlParams.get('file');
5   if (!file) {
6     const match = window.location.pathname.match(/\/([^\/] +\.html)$/);
7     if (match) file = match[1];
8   }
9
10  fetch('/article_fetcher.php?file=${encodeURIComponent(file)}')
11    .then(response => response.text())
12    .then(html => {
13      const parser = new DOMParser();
14      const doc = parser.parseFromString(html, 'text/html');
15
16      // Convert all headings to h4
17      doc.querySelectorAll('h1,h2,h3,h5,h6')
18        .forEach(h => { /* convert to h4 */ });
19
20      // Normalize relative paths to absolute
21      doc.querySelectorAll('img').forEach(img => {
22        if (img.src && !img.src.startsWith('http')) {
23          img.src = 'https://www.orthodox.net/articles/' +
24            img.src;
25        }
26      });
27
28      document.getElementById('article-placeholder')
29        .innerHTML = doc.body.innerHTML;
30    });
31});
```

Sermon-specific cleaning (`sermon-shell.js`) additionally:

- Removes Microsoft Word CSS classes: `MsoBodyText`, `MsoFootnoteText`, `MsoHeader`
- Strips all `<style>` tags and inline style attributes
- Converts `p.MsoHeader` elements to semantic heading tags
- Normalizes media file links (jpg, mp3, pdf) to absolute URLs at `https://www.orthodox.net/sermons/`

5 Frontend Architecture

5.1 HTML Template Structure

All pages share a common template maintained by `util/update_template.py`. The template (`util/template.html`) defines the HTML `<head>` section:

- Meta tags (charset, viewport, CSP, description, keywords, OpenGraph)
- CSS imports (Skeleton, custom stylesheets, icon fonts)
- Analytics and PayPal SDK scripts

The body structure across all pages:

```
<div class="section section-banner">
  <!-- Church logo, title, subtitle -->
</div>
<div class="navBar-section" id="navbar">
  <!-- Navigation links, hamburger menu -->
</div>
<div class="section section-main" id="page-content">
  <!-- Page-specific content -->
</div>
<div class="section section-other-links">
  <!-- Footer links -->
</div>
```

5.2 CSS Architecture

5.2.1 Skeleton Framework

The site uses Skeleton CSS (`/css/skeleton.css`), a lightweight responsive grid system with a 12-column layout. Skeleton provides base typography and a mobile-first responsive grid.

5.2.2 Custom Stylesheets

File	Version	Purpose
<code>style.css</code>	?v=14	Primary site styles: colors, typography, section layout
<code>nav.css</code>	—	Navigation bar: desktop float layout, mobile hamburger
<code>calendar.css</code>	—	Holy Trinity calendar content styling
<code>normalize.css</code>	—	Browser reset (bundled with Skeleton)
<code>dropdown.css</code>	—	Dropdown menu styling

5.2.3 Design Tokens

Key values from `style.css`:

Property	Value	Usage
Background	#f2e1c5	Page background (cream)
Link color	#1c5880	All anchor tags (dark blue)
Banner title	#850f00	Church name heading (dark red)
Banner subtitle	#1c5880	Subtitle text (dark blue)
Nav background	darkslategray	Navigation bar (opacity 0.8)
Nav hover	#009cde text on white bg	Navigation link hover state
Base font size	75%	HTML root element
Font family	Alegreya, serif	Primary typeface (Google Fonts)

5.2.4 Responsive Breakpoints

Breakpoint	Behavior
$\leq 550\text{px}$	Video thumbnails shrink to 160px width
$\leq 700\text{px}$	Hamburger menu activates; nav links collapse; video thumbnails 260px
$\leq 900\text{px}$	Video thumbnails reduce to 320px width
$> 700\text{px}$	Full horizontal navigation bar; video thumbnails at 400px

5.3 JavaScript Architecture

Figure 8 shows how the homepage JavaScript files connect to APIs and DOM elements.

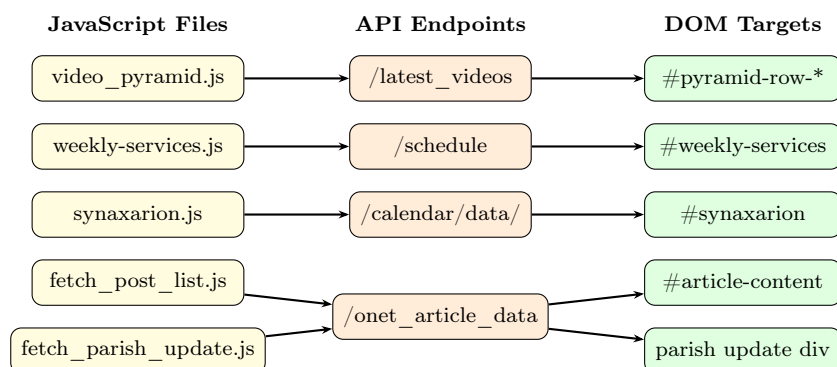


Figure 8: Homepage: JavaScript files, API endpoints, and DOM injection targets.

There is no build system, bundler, or transpiler. All JavaScript files are vanilla ES5/ES6 loaded via `<script>` tags with query-string cache busting (`?v=N`).

5.3.1 Configuration

Listing 3: /content/js/config.js

```
1 const CONFIG = {  
2   VERSION: '2',  
3   FLASK_BACKEND_URL:  
4     'https://production-orthodox-net-flask.onrender.com'  
5 };
```

All scripts that call the backend API reference `CONFIG.FLASK_BACKEND_URL`. This file **must** be included before any API-calling script:

```
<script src="/js/config.js?v=2"></script>  
<script src="/js/fetch_article.js?v=1"></script>
```

5.3.2 Page-Level Cache Busting

The homepage (`index.html`) implements a page-level cache buster via `localStorage`:

```
const PAGE_VERSION = '20241202a';  
if (localStorage.getItem('pageVersion') !== PAGE_VERSION) {  
  localStorage.setItem('pageVersion', PAGE_VERSION);  
  window.location.reload(true);  
}
```

Format: `YYYYMMDDx` where `x` is a letter suffix for same-day updates.

5.3.3 JavaScript File Reference

Core/Infrastructure

File	Purpose
config.js	API base URL and version constant
navbar.js	Hamburger menu toggle; listens on .icon click, toggles .responsive class on #mainNavBar
gtag.js	Google Analytics initialization
ppal.js	PayPal donation form integration
analytics.js	(Deprecated/empty)

Homepage Dynamic Content

File	API Endpoint	Target DOM Element
video_pyramid.js	/latest_videos	#pyramid-row-1 through #pyramid-row-5
weekly-services.js	/schedule	#weekly-services
synaxarion.js	Local file	#synaxarion
fetch_post_list.js	/onet_article_data	#article-content

fetch_parish_update.js	/	Parish update section
	onet_article_data	

CMS Content Scripts

File	API Endpoint	Target DOM
fetch_article.js	/onet_article	#article-content
fetch_all_posts.js	/onet_article_data	Post listing
fetch_article_list.js	/onet_articles_to_list	Article listing
fetch_article_by_slug.js	/onet_article	Article content
fetch_latest_article.js	/onet_article_data	Latest article
fetch_post.js	/onet_article	#article-content
fetch_motd.js	/onet_article_data	Message of the day
fetch_parish_life.js	/parish_life	Parish life section
show-legacy-article.js	/legacy_article	Legacy article display
fetch_legacy_article.js	/legacy_article	Legacy content
fetch_legacy_article_from_cms.js	/legacy_article	Legacy via CMS

Legacy Folder Shell Scripts (22 files) All *-shell.js files follow the pattern described in Section 4.3. Each corresponds to a legacy content category and its PHP fetcher.

Specialized Scripts

File	Purpose
catechism.js	Catechism page interactivity
catechism-qa.js	Catechism Q&A accordion/toggle behavior
carousel_main.js	YouTube video carousel (homepage variant)
format-bible-verses.js	Formats inline Bible verse references
gleanings-categories.js	Gleanings content category filtering
gleanings-topics.js	Gleanings topic-based navigation
change-url.js	URL manipulation utility
change-blog-url.js	Blog URL formatting
content-html.js	Generic content HTML processing
blog.js	Blog page functionality
calendar.js	Calendar page display
homilies.js	Homilies listing
latest-homily.js	Latest homily display
loadCalendar2.js	Alternate calendar loader

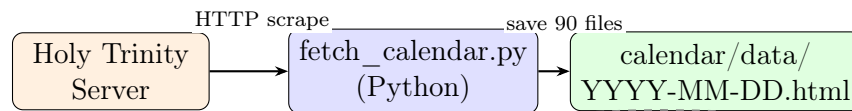
6 Synaxarion (Saints & Readings)

The “Today’s Saints & Readings” section on the homepage uses pre-generated static HTML files rather than a live API call, for performance. Figure 9 illustrates the two-phase data flow.

6.1 Data Source

Calendar data is scraped from Holy Trinity Russian Orthodox Church:

Phase 1: Generation



Phase 2: Serving

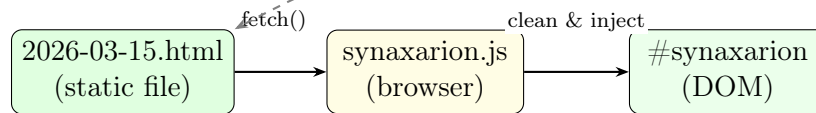


Figure 9: Synaxarion data flow: pre-generation (offline) and serving (runtime).

```
https://www.holytrinityorthodox.com/calendar/doc/examples/ppp.php
?month=MM&today=DD&year=YYYY
&dt=1&header=1&lives=1&trp=1&scripture=1
&sid=RANDOM
```

6.2 Data Files

Files are stored at `/content/calendar/data/YYYY-MM-DD.html`. Each file contains the saints, scripture readings, and troparion for that date.

6.3 Client-Side Loading (`synaxarion.js`)

1. Computes today's date in the browser
2. Constructs path: `/calendar/data/YYYY-MM-DD.html`
3. Fetches the file via `fetch()`
4. Parses response as HTML, removes `.ptroparionheader` and everything after it
5. Sets `alt=""` on `jcal_img` images for accessibility
6. Converts all `http://` URLs to `https://`
7. Sets Holy Trinity links to `target="_blank"` with `rel="noopener"`
8. Inserts into `#synaxarion` div

6.4 Maintenance

Critical: The `fetch_calendar.py` script generates files for 90 days from the run date. If the script is not re-run periodically, the homepage will display a 404 error in the saints/readings section. **Run at least every 90 days** on both local and production:

```
# Local
cd util
python3 fetch_calendar.py

# Production (via cPanel terminal)
```

```
cd /public_html/new/onet-v2/util
python3 fetch_calendar.py
```

The generated files are committed to git so they deploy with `git pull`, but the production server also needs its own run since the local and production calendars may drift.

7 Backend API Reference

The Flask backend is in a separate repository: <https://github.com/GZancewicz/onet-flask-backend>. This section documents the endpoints as consumed by the frontend.

7.1 Base URL

```
Production: https://production-orthodox-net-flask.onrender.com
Staging:    https://staging-orthodox-net-flask.onrender.com
Legacy:     https://onet-flask-backend-1f4c6a3f01c8.herokuapp.com
```

Configured in `/content/js/config.js`. The `.env` file documents all URLs but is not read at runtime (static site has no build process).

7.2 Endpoints

Endpoint	Method	Description
<code>/latest_videos</code>	GET	Returns JSON array of latest YouTube videos (title, url, published_at, thumbnail)
<code>/schedule</code>	GET	Returns JSON array of upcoming services (date, time, description)
<code>/onet_article_data</code>	GET	Returns all articles with metadata (title, posted_at, tags, url, id)
<code>/onet_article</code>	GET	Returns single article by ID (full HTML content)
<code>/onet_articles_to_list</code>	GET	Returns filtered article list (orthodox_net tag)
<code>/parish_life</code>	GET	Returns articles tagged <code>parish_life</code>
<code>/legacy_article</code>	GET	Returns legacy article content (requires <code>filename</code> param)
<code>/heartbeat</code>	GET	Health check endpoint

8 Python Utilities

All utility scripts are in `/util/`. Install dependencies first:

```
cd util
pip install -r requirements.txt
```

8.1 Content Generation Scripts

Script	Details
<code>fetch_calendar.py</code>	Fetches saints/readings from Holy Trinity Orthodox for the next 90 days. Saves to <code>../content/calendar/data/YYYY-MM-DD.html</code> . Uses random sid parameter for cache bypass. Must be run periodically. Usage: <code>python3 fetch_calendar.py</code>
<code>fetch_youtube.py</code>	Fetches latest 100 videos from @orthodoxnet YouTube channel (ID: UCb_MxJmD_J4peQtRHj-qBKA). Uses YouTube Data API v3 (search + videos endpoints). Requires API key from <code>youtube_credentials.py</code> . Exports to <code>../content/videos.json</code> . Usage: <code>python3 fetch_youtube.py</code>

8.2 Template and Styling Scripts

Script	Details
<code>update_template.py</code>	Reads <code>template.html</code> and replaces the <code><head></code> section of every <code>.html</code> file under <code>../content/</code> (excluding files starting with <code>_</code>). Overwrites files in place with UTF-8 encoding. Usage: <code>python3 update_template.py</code>
<code>update_css_version.py</code>	Updates CSS version query strings across HTML files for cache busting. Usage: <code>python3 update_css_version.py</code>

8.3 Legacy HTML Processing

Script	Details
<code>clean_html_file.py</code>	Sanitizes a single HTML file: removes accented characters (á, ô, ö, û, æ, etc.), strips legacy header images, removes OIS attribution paragraphs. Processes only content within <code><div id="placeholder"></code> . Usage: <code>python3 clean_html_file.py input.html output.html</code>
<code>clean_all_html_files.py</code>	Batch version of <code>clean_html_file.py</code> . Processes all HTML files in a directory. Usage: <code>python3 clean_all_html_files.py folder_path/</code>
<code>strip_attributes.py</code>	Removes all HTML comments and strips attributes from tags (except <code><a></code> tags) from an input file. Usage: <code>python3 strip_attributes.py input.html output.html</code>

<code>strip_body.py</code>	Extracts content inside the <code><body></code> tag and writes to output file. Usage: <code>python3 strip_body.py input.html output.html</code>
<code>replace_body_content.py</code>	Replaces the body content of an HTML file with new content. Usage: <code>python3 replace_body_content.py input.html</code>
<code>fix_email_spacing.py</code>	Fixes whitespace/formatting issues around email addresses in HTML. Usage: <code>python3 fix_email_spacing.py</code>

8.4 File Management and Conversion

Script	Details
<code>png2webp.py</code>	Converts PNG images to WebP format using Pillow. Usage: <code>python3 png2webp.py input.png output.webp</code> If output path omitted, generates from input filename.
<code>crawl_directory.py</code>	Crawls a directory, catalogs files with modification times, extracts unique filenames, creates <code>/text</code> directory, converts <code>.doc</code> files to <code>.txt</code> via LibreOffice. Usage: <code>python3 crawl_directory.py -folder FOLDERNAME</code>
<code>convert_journal_index.py</code>	Converts journal index content for the journal section. Usage: <code>python3 convert_journal_index.py</code>
<code>identify_pdffonts.py</code>	Analyzes and reports fonts used in PDF files. Usage: <code>python3 identify_pdffonts.py filename.pdf</code>

8.5 Gleanings Utilities

Located in `/util/gleanings-utils/`. These scripts process and organize the “Gleanings of the Holy Fathers” content:

- `rebuild_gleanings_from_legacy.py` – Rebuilds gleanings data from legacy HTML
- `extract_gleanings_topics.py` – NLP-based topic extraction
- `generate_sub_all_topics.py` – Generates topic index
- `add_father_field.py` – Adds Church Father attribution
- `add_scripture_field.py` – Adds scripture references
- `fix_authors_and_names.py` – Normalizes author names
- `fix_json_titles.py` (v1–v4) – Iterative title cleanup

- `fix_orthodox_book_names.py` – Standardizes book name references
- `change_author_to_person.py` – Schema field rename

8.6 NLP Utilities

Located in `/util/nlp/`:

- `extract_topics.py` (v1, v2, modified variants) – Topic extraction from religious texts using NLTK and spaCy

8.7 Dependencies (`requirements.txt`)

Package	Purpose
<code>requests >= 2.28.0</code>	HTTP requests (calendar fetching, API calls)
<code>beautifulsoup4 >= 4.11.0</code>	HTML parsing and manipulation
<code>Pillow >= 9.0.0</code>	Image format conversion (PNG to WebP)
<code>PyPDF2 >= 2.0.0</code>	PDF font analysis
<code>nltk >= 3.7.0</code>	Natural language processing (topic extraction)
<code>spacy >= 3.2.0</code>	NLP pipeline (entity recognition)
<code>google-api-python-client >= 2.0.0</code>	YouTube Data API client
<code>google-auth-oauthlib >= 0.4.0</code>	Google API authentication
<code>selenium >= 4.1.0</code>	Browser automation (web scraping)
<code>webdriver-manager >= 3.8.0</code>	Selenium WebDriver management
<code>pathlib >= 1.0.1</code>	Path manipulation

9 Local Development

9.1 Setup

1. Clone the repo:

```
git clone https://github.com/GZancewicz/onet-v2.git
```

2. Install Python dependencies:

```
cd onet-v2/util
pip install -r requirements.txt
```

3. Generate calendar data:

```
python3 fetch_calendar.py
```

4. For legacy content testing, clone the legacy pages repo into a sibling `public_html/` directory: <https://github.com/GZancewicz/orthodox-net-2023>

Simulating the production server locally: The development environment mirrors the production directory structure by wrapping the `onet-v2` clone inside a `public_html/new/` folder hierarchy. This allows the relative paths used by PHP

fetcher scripts (e.g., `__DIR__ . '/../../../../../public_html/sermons/'`) to resolve correctly on both local and production. You do not need all legacy files locally — just enough to test that the `.htaccess` routing and PHP fetchers work for the pages you are developing. The `apache/` folder in the repo contains reference Apache configuration files, including a local `httpd.conf` that points DocumentRoot to the `content/` directory.

9.2 Running Locally

9.2.1 Option A: PHP Built-in Server (Recommended)

The simplest approach. Supports PHP fetcher scripts for legacy content rendering. The `content/.htaccess` file provides shell routing with relative paths.

```
cd content
php -S localhost:8000
```

Open <http://localhost:8000> in your browser. The homepage, CMS content, and calendar will work. Legacy content fetchers will work if your directory structure has legacy files at the expected relative paths.

9.2.2 Option B: VSCode LiveServer

Quick and easy for static content, but PHP fetcher scripts will **not** execute. Legacy content shell pages will show errors. Useful for working on CSS, layout, or CMS-driven content only.

9.2.3 Option C: Local Apache (Full Simulation)

For complete production simulation including `mod_rewrite` rules. Use the provided `apache/httpd.conf` which is configured for macOS with Homebrew Apache. It sets DocumentRoot to the `content/` directory and listens on port 8080.

```
# macOS with Homebrew Apache
sudo apachectl -f /path/to/onet-v2/apache/httpd.conf -k start
```

Note: The provided `httpd.conf` has `mod_rewrite` commented out. You must uncomment it for `.htaccess` rewrite rules to work:

```
# Uncomment this line in httpd.conf:
LoadModule rewrite_module /opt/homebrew/lib/httpd/modules/mod_rewrite.
so
```

You will also need to set `AllowOverride All` (instead of `None`) in the `<Directory>` block for `.htaccess` files to be processed.

9.3 Local vs. Production Differences Summary

Aspect	Local (php -S)	Production (Inoa)
Server	PHP built-in server	Apache 2.x with cPanel
DocumentRoot	<code>content/</code>	<code>/public_html/</code>
<code>.htaccess</code>	<code>content/.htaccess</code> (relative paths)	<code>/public_html/.htaccess</code> (absolute paths)

.htaccess managed by	Git (in repo)	Manual (cPanel)
PHP version	System PHP	ea-php82 (cPanel)
Legacy files at	Sibling public_html/ dir	/public_html/CATEGORY/
mod_rewrite	Not available	Active
URL normalization	Not active	Lowercase enforcement
Media CDN	Not active	Redirects to media.orthodox.net

10 Production Deployment

10.1 Standard Deployment

Via cPanel terminal on the Inoa server:

```
cd /public_html/new/onet-v2
git pull
```

Requires GitHub username and personal access token for authentication.

What git pull deploys: All files in the repository — HTML pages, JavaScript, CSS, PHP fetchers, Python scripts, calendar data, images, documentation.

What git pull does NOT deploy: The production .htaccess at ~/public_html/.htaccess. Maintain separately (Section 3).

10.2 Template Updates

After modifying util/template.html, propagate changes to all pages:

```
cd util
python3 update_template.py
```

Then commit the updated HTML files and deploy.

10.3 Calendar Data Refresh

```
cd /public_html/new/onet-v2/util
python3 fetch_calendar.py
```

This generates files for 90 days. Set a reminder to re-run quarterly at minimum.

11 Security

11.1 Content Security Policy

The CSP header is set in the HTML template <meta> tag:

```
<meta http-equiv="Content-Security-Policy"
      content="script-src 'self'
              https://www.googletagmanager.com
              'sha256-wXSv6GR1S4j0QzLWBcHDAeshyZPL0rZHR0tUYVbwfRc='>
```

```
https://www.paypal.com
'sha256-MrcTPuM3qtLGdi6sqI03YzkTqvRvqCp30Plfs/1JYyE='
https://www.sandbox.paypal.com">
```

Scripts are whitelisted by origin or SHA-256 hash. Inline scripts require a matching hash.

11.2 Directory Traversal Prevention

All PHP fetcher scripts use `basename()` on the `file` query parameter, ensuring only filenames (not paths) are accepted. This prevents attackers from reading arbitrary files via path traversal (e.g., `?file=../../etc/passwd`).

11.3 HTTPS Enforcement

- All external resources loaded over HTTPS
- `synaxarion.js` converts any `http://` URLs in calendar data to `https://`
- Fetcher PHP scripts do not expose `http://` content

11.4 Credentials and Secrets

These files contain sensitive data and are listed in `.gitignore`:

File	Contents
<code>.env</code>	Backend API URLs (local documentation only, not read at runtime)
<code>youtube_credentials.py</code>	YouTube Data API key (function <code>get_api_key()</code>)
<code>credentials</code>	Git credentials for server deployment
<code>github.pat</code>	GitHub personal access token

If you are taking over this project, you will need to obtain these credentials from the current maintainer or regenerate them from the respective services.

12 Character Encoding

The codebase handles three encoding contexts:

Context	Encoding	Notes
Legacy HTML files	Windows-1252	Declared via <code>.htaccess</code> <code>Ad-dType</code> directive (<code>windows-1251</code> is declared but files are actually Windows-1252)
PHP fetchers	UTF-8 output	Convert from Windows-1252 via <code>mb_convert_encoding()</code>

The `clean_html_file.py` utility also strips specific accented characters that cause rendering issues: á, Â, ô, Ô, ö, Ö, û, Û, æ, Æ.

13 Cache Busting Strategy

Three levels of cache busting are employed:

1. **JavaScript/CSS files:** Query string versioning on `<script>` and `<link>` tags (e.g., `style.css?v=14`, `config.js?v=2`). Increment manually when files change.
2. **Page-level:** `PAGE_VERSION` constant in `index.html` triggers `localStorage`-based forced reload when updated. Format: `YYYYMMDDx`.
3. **API config:** `CONFIG.VERSION` in `config.js` can be used by scripts to append version info to API requests.

Workflow when updating assets:

1. Edit the CSS or JS file
2. Increment the `?v=` query string in all HTML files that reference it
3. If updating `config.js`, bump `CONFIG.VERSION`
4. Optionally bump `PAGE_VERSION` in `index.html` to force page reload
5. Commit all changes and deploy via `git pull`

`update_css_version.py` can automate CSS version bumps across files.

14 Key DOM Element IDs

Reference for DOM elements manipulated by JavaScript:

Element ID	Page	Content
<code>#navbar</code>	All	Navigation bar section
<code>#mainNavBar</code>	All	Navigation link container
<code>#page-content</code>	All	Main content wrapper
<code>#weekly-services</code>	Homepage	Upcoming services table
<code>#synaxarion</code>	Homepage	Saints & readings content
<code>#video-pyramid</code>	Homepage	YouTube video grid
<code>#pyramid-row-1 ... -5</code>	Homepage	Individual video rows
<code>#article-content</code>	Homepage/Posts	Blog post listing or article body
<code>#article-placeholder</code>	Article shell	Legacy article content injection

<code>#sermon-placeholder</code>	Sermon shell	Legacy sermon content injection
<code>#donate</code>	Homepage	PayPal donation forms

15 Troubleshooting

15.1 “Not Found” in Saints & Readings Section

The synaxarion section shows a 404 error when calendar data files have expired. Run `fetch_calendar.py` on both local and production (see Section 6.4).

15.2 Legacy Content Not Rendering

1. Verify the `.htaccess` shell routing rule exists for the content category
2. On production, check `~/public_html/.htaccess` (not the repo version)
3. Verify the PHP fetcher exists at `content/CATEGORY_fetcher.php`
4. Verify the legacy HTML file exists at `/public_html/CATEGORY/filename.html`
5. Check that the legacy file contains `<!--BEGIN-CONTENT-->` markers

15.3 Backend API Errors

1. Check the API health: visit `CONFIG.FLASK_BACKEND_URL + '/heartbeat'`
2. If Render.com free tier, the service may be sleeping (cold start takes 30–60 seconds)
3. Verify `config.js` has the correct `FLASK_BACKEND_URL`
4. Check browser console for CORS errors

15.4 Changes Not Appearing After Deploy

1. Clear browser cache or hard-refresh (Ctrl+Shift+R)
2. Bump the `?v=` query string on changed files
3. Bump `PAGE_VERSION` in `index.html`
4. On production, verify `git pull` completed successfully
5. Remember: `.htaccess` changes require manual server update

16 Repository Files Reference

16.1 Git-Ignored Files

The following files are excluded from version control (`.gitignore`):

```
.gitignore           # Self-reference
httpd.conf           # Local Apache config
github.pat           # GitHub personal access token
credentials           # Git credentials
/.vscode             # IDE settings
youtube_credentials.py # YouTube API key
.DS_Store            # macOS metadata
settings.local.json  # Local settings
*.aux, *.log, *.out, *.synctex # LaTeX build artifacts
.env                 # Environment variables
.claude              # AI assistant data
```

16.2 Documentation

- `README.md` – Project overview and contributor guide
- `CLAUDE.md` – AI coding assistant instructions
- `doc/latex/onet_overview.tex` – Platform architecture overview
- `doc/latex/onet_frontend.tex` – This document
- `content/convert-legacy-folder.md` – Guide for converting legacy content folders

17 Handoff Checklist

If you are taking over maintenance of this project, ensure you have:

1. **GitHub access:** Write access to <https://github.com/GZancewicz/onet-v2> and <https://github.com/GZancewicz/onet-flask-backend>
2. **Inoa/OIS cPanel access:** Login credentials for the hosting control panel (manage files, terminal, PHP version, etc.)
3. **Ghost CMS access:** Admin login for the Ghost.io instance (article/post management)
4. **Google API credentials:** YouTube Data API key (in `youtube_credentials.py`)
5. **Render.com access:** Dashboard access for the Flask backend (logs, environment variables, deploys)
6. **Google Analytics:** Access to the G-D9YEGT19W7 property
7. **PayPal:** Access to the PayPal business account for donation configuration
8. **DNS:** Understanding of who manages DNS for `orthodox.net` (currently Inoa/OIS)

9. **Production .htaccess:** A current copy of `~/public_html/.htaccess` from the server (compare with `.htaccess.production` in the repo)
10. **Calendar cron:** Set a recurring reminder to run `fetch_calendar.py` at least every 90 days on the production server

A Project History by Milestone

This appendix summarizes the development history of `onet-v2` organized by major milestone. The full commit history is available via `git log` on the `main` branch at <https://github.com/GZancewicz/onet-v2>.

A.1 Phase 1: Initial Setup and Infrastructure (Jul–Aug 2023)

The project began with an initial commit on July 30, 2023. This phase established the foundational infrastructure for the new site.

- Created the repository and deployed the first “Hello World” test to the Inoa server
- Set up the directory structure (`content/`, `util/`, `legacy/`) and configured Apache `.htaccess` routing to serve new content while falling back to legacy pages
- Built the homepage with the Skeleton CSS framework, Alegreya font, and responsive navigation
- Integrated Google Analytics and PayPal donation forms
- Created the first content pages: `/articles/index.html`, `/sermons/index.html`
- Developed Python utility scripts: `update_template.py` (head section updater), `png2webp.py` (image converter), `strip_attributes.py`, `strip_body.py`
- Added the “About Us” section with address, phone, directions, and contact information

A.2 Phase 2: Dynamic Content Integration (Aug–Oct 2023)

Connected the site to external content sources via the Flask backend API.

- Integrated YouTube video carousel on the homepage, initially fetching directly from the YouTube API, then migrated to the Heroku-hosted Flask backend
- Built the synaxarion (saints and readings) section with `fetch_calendar.py` scraping Holy Trinity Orthodox calendar data into static HTML files, and `synaxarion.js` loading them client-side
- Added weekly services schedule display via the Flask API’s `/schedule` endpoint
- Created the catechism section with Q&A interactive elements
- Built the catechesis pages with legacy content integration
- Began initial Ghost CMS integration testing for article fetching

A.3 Phase 3: CMS and Blog System (Dec 2024–Feb 2025)

Established the Ghost CMS as the primary content management system for new articles and blog posts.

- Implemented blog post display with meaningful URLs (`/posts/slug`)
- Built article listing and individual article pages fetching from Ghost CMS via the Flask API
- Added tag-based filtering, article counts, and formatting improvements
- Added pagination to the All Posts page
- Created the parish life section (filtered by `parish_life` tag)
- Added message of the day feature (filtered by `motd` tag)
- Implemented legacy article display through the CMS
- Reorganized homepage sections: moved parish appeal panel, added parish life links
- Fixed numerous Ghost CMS rendering issues (inline images, feature images, download icons, image card formatting)

A.4 Phase 4: Legacy Folder Migration System (Jun–Aug 2025)

The largest development phase — built the system for serving legacy HTML content through the new site template. This introduced the PHP content fetchers, JavaScript shell scripts, and `.htaccess` rewrite rules that form the legacy migration architecture (Section 4).

- Developed the PHP fetcher pattern: extract content between content markers, convert Windows-1252 to UTF-8, serve via AJAX
- Created the JavaScript shell script pattern: fetch from PHP, clean Word artifacts, normalize paths, inject into DOM
- Built the `content/.htaccess` routing rules (local) and the corresponding production version for the Inoa server
- Migrated 24 legacy content folders in rapid succession:
 - `/articles`, `/sermons` (June 2025)
 - `/scripture`, `/full-voice`, `/questions`, `/greatlent` (June–July 2025)
 - `/pascha`, `/theophany`, `/confess`, `/fathers` (July 2025)
 - `/gleanings`, `/ustav`, `/services`, `/trebnic` (July 2025)
 - `/music`, `/menaion-thoughts`, `/saints`, `/russianm` (July 2025)
 - `/links`, `/western-saints`, `/monastic-stories`, `/poetry` (July 2025)
 - `/nativity` (July 2025)
 - `/journal` (November 2025)

- Fixed character encoding issues with Russian and accented characters
- Created `clean_html_file.py` and `clean_all_html_files.py` for batch legacy HTML sanitization
- Developed the gleanings subsystem with NLP-based topic extraction, category filtering, and topic navigation
- Added LaTeX technical documentation

A.5 Phase 5: API Migration and Stabilization (Nov–Dec 2025)

Migrated the backend API from Heroku to Render.com and stabilized the site for production use.

- Migrated Flask backend from Heroku to Render.com (both staging and production environments)
- Introduced `config.js` as a centralized API URL configuration file, replacing hard-coded URLs across JavaScript files
- Implemented query-string cache busting scheme (`?v=N`) across all JS and CSS files
- Reorganized the homepage video display from carousel to video pyramid layout
- Fixed numerous cache-related issues on production
- Updated production `.htaccess` to include all 24 legacy folder routing rules
- Upgraded the All Posts page
- Fixed URL formatting issues impacting post display

A.6 Phase 6: Documentation and Maintenance (2026–Present)

Ongoing documentation, maintenance, and operational improvements.

- Comprehensive rewrite of this technical reference document with TikZ diagrams
- Updated `README.md` to reflect current architecture
- Identified and resolved the calendar data expiration issue (saints & readings 404)
- Reorganized `doc/latex/` into per-document subdirectories

B Document Revision History

This appendix records the revision history of this document based on the git commit history.

Date	Commit	Description
2025-04-04	c005113	Initial creation. Added technical description files covering platform overview, frontend, and backend as three separate LaTeX documents.
2025-07-13	4b2dae0	Added LaTeX documentation with expanded content covering the legacy folder migration system.
2025-11-27	b0ae272	Deleted obsolete LaTeX documentation for a planned rewrite.
2025-11-30	5326e0d	Reorganized LaTeX files. Restored frontend and overview documents. Removed the standalone backend document (content merged into frontend).
2025-11-30	20ea1ea	Moved files into reorganized directory structure.

2026-03-15	(this session)	Comprehensive rewrite of this document. Major changes: • Expanded from 4 pages to 32+ pages • Added 8 TikZ diagrams (system architecture, tech stack, external services, routing flowchart, local vs. production paths, legacy migration sequence, homepage JS architecture, synaxarion data flow) • Added new sections: legacy LAMP stack origins and project goals; production <code>.htaccess</code> management (local vs. production); legacy content migration request flow with code examples; synaxarion maintenance; troubleshooting guide; handoff checklist • Added Appendix A: project history by milestone (6 phases) • Added Appendix B: this revision history • Corrected production server architecture: single <code>.htaccess</code> at <code>/public_html/</code>, not at <code>/public_html/new/</code> • Documented that <code>.htaccess</code> files are not deployed via <code>git pull</code> and must be manually maintained • Documented GitHub repository URL • Reorganized into per-document subdirectories under <code>doc/latex/</code> • Deleted standalone backend document (<code>onet_backend.tex</code>) • Resolved all overfull hbox and vbox warnings
------------	----------------	---
